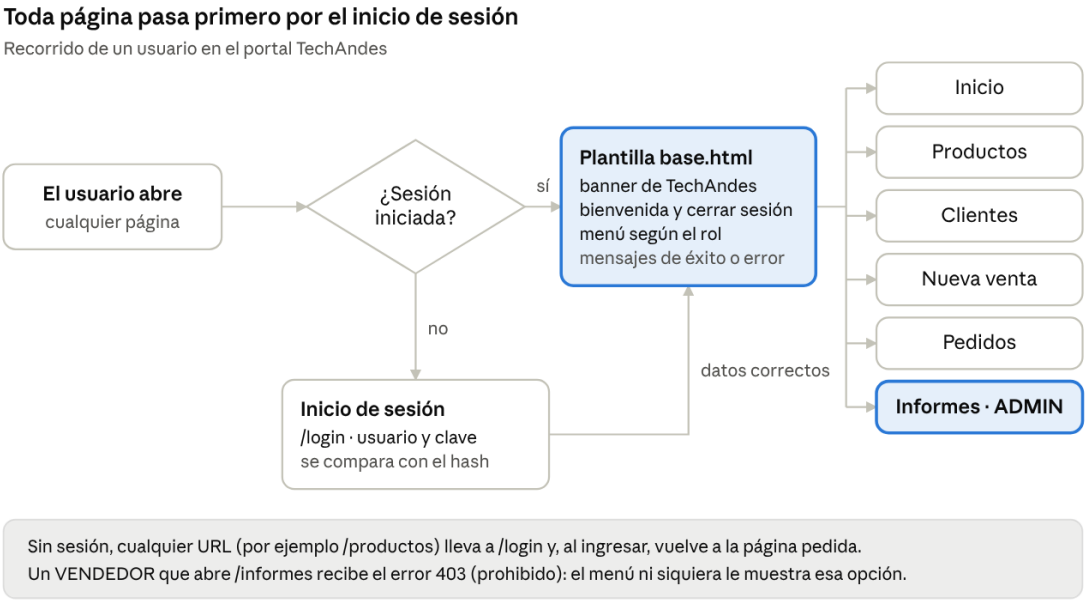
Taller Python: Portal web TechAndes con Flask

Nov, 2026 · @Luis Galicia

1. Qué vamos a construir

Construirán en Python, con el framework **Flask**, el portal interno de ventas de TechAndes sobre la base `techandes` de MySQL. Toma la estructura del portal del videoclub (banner, barra de bienvenida con *Cerrar sesión*, menú y área de contenido) y la lleva a un diseño actual, con inicio de sesión real, roles y operaciones sobre la base de datos.



Recorrido del usuario: inicio de sesión, plantilla y módulos

TechAndes:

Portal TechAndes	Qué hace
`/login`	Valida usuario y contraseña contra la tabla `usuarios`
`banner.svg` con logo y cordillera	Identidad visual, igual en todas las páginas
Nombre, cargo y rol del empleado + *Cerrar sesión*	Muestra quién está conectado

Portal TechAndes	Qué hace
Inicio	Indicadores, últimos pedidos y productos por reabastecer
Productos	Catálogo con búsqueda; el ADMIN crea, edita y elimina
Clientes	Búsqueda, registro e historial de compras
Nueva venta	Registra un pedido con transacción y descuenta el stock
Pedidos	Cambia el estado; al cancelar, devuelve las unidades al inventario
Informes (solo ADMIN)	Ventas por categoría, ranking de vendedores, top de productos

Tecnologías: Python 3.10 o superior, Flask 3 (servidor web y rutas), Jinja2 (plantillas HTML, viene con Flask), PyMySQL (conexión con MySQL) y python-dotenv (configuración). No se usa JavaScript: todo se procesa en el servidor.

.

Usuarios de prueba (creados en el paso 2):

Usuario	Empleado	Rol	Contraseña
`mprieto`	Martha Lucía Prieto Gaitán (gerente)	ADMIN	`Admin2026*`
`hbustos`, `crobayo`	Supervisores de ventas	ADMIN	`Admin2026*`
`klozada`, `ytafur`, `boviedo`, `lmurcia`, `fcuellar`	Asesores comerciales	VENDEDOR	`Ventas2026*`

2. Paso 1: preparar el entorno de Python

1.1 Instalar Python. Descargue Python 3.12 o superior desde python.org. En Windows, en la primera pantalla del instalador marque **"Add python.exe to PATH"**; si se olvida, la terminal no reconocerá el comando `python`. Verifique:

```
python --version
```

1.2 Editor. Visual Studio Code con la extensión **Python** (de Microsoft). Opcional: la extensión *Jinja* para resaltar las plantillas.

1.3 Crear la carpeta y el entorno virtual

```
mkdir techandes-portal
cd techandes-portal
python -m venv .venv
```

Un **entorno virtual** (`.venv`) es una copia aislada de Python solo para este proyecto. Lo que se instale aquí no afecta otros proyectos del computador, y cada proyecto puede usar versiones distintas de las librerías.

1.4 Activar el entorno. Se hace cada vez que se abre una terminal nueva:

```
# Windows (PowerShell o CMD)
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

Cuando está activo, la terminal muestra `(.venv)` al inicio de la línea. En VS Code: `Ctrl + Shift + P` → *Python: Select Interpreter* → elegir el de `.venv`.

Si PowerShell responde *"la ejecución de scripts está deshabilitada en este sistema"*, ejecute una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` o use CMD.

1.5 Instalar las librerías. Cree el archivo `requirements.txt`:

```
Flask>=3.0
PyMySQL>=1.1
python-dotenv>=1.0
cryptography>=42.0
```

y ejecute:

```
pip install -r requirements.txt
```

Librería	Para qué sirve
`Flask`	Framework web: recibe las peticiones del navegador, ejecuta la función de cada ruta y devuelve HTML. Trae **Jinja2** (plantillas) y **Werkzeug** (sesiones y cifrado de contraseñas)
`PyMySQL`	Conector de Python con MySQL, escrito en Python puro (no requiere compilar nada)
`python-dotenv`	Lee el archivo `.env` con la configuración
`cryptography`	La necesita PyMySQL para el método de autenticación de MySQL 8 (`caching_sha2_password`)

`requirements.txt` es la lista de dependencias del proyecto: quien descargue el código instala todo con un solo comando.

3. Paso 2: tabla de usuarios y contraseñas cifradas

Para iniciar sesión hace falta una tabla nueva, `usuarios`, enlazada con `empleados`: cada empleado tiene a lo sumo un usuario, y cada usuario es un empleado. Así el portal sabe quién registra cada venta.

2.1 La tabla (archivo `sql/01_usuarios.sql`, se ejecuta en Workbench como `root`):

```
USE techandes;

CREATE TABLE usuarios (
  id_usuario     INT          NOT NULL AUTO_INCREMENT,
  usuario        VARCHAR(30)  NOT NULL,
  clave_hash     VARCHAR(255) NOT NULL,
  rol            VARCHAR(15)  NOT NULL DEFAULT 'VENDEDOR',
  id_empleado    INT          NOT NULL,
  activo         TINYINT      NOT NULL DEFAULT 1,
  ultimo_acceso  DATETIME     NULL,
  CONSTRAINT pk_usuarios PRIMARY KEY (id_usuario),
  CONSTRAINT uq_usuarios_usuario  UNIQUE (usuario),
  CONSTRAINT uq_usuarios_empleado UNIQUE (id_empleado),
  CONSTRAINT fk_usuarios_empleados FOREIGN KEY (id_empleado)
      REFERENCES empleados (id_empleado)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT ck_usuarios_rol    CHECK (rol IN ('ADMIN', 'VENDEDOR')),
  CONSTRAINT ck_usuarios_activo CHECK (activo IN (0, 1))
) ENGINE = InnoDB;
```

El `UNIQUE` sobre `id_empleado` convierte la relación en **1:1**: sin él, un empleado podría tener varios usuarios. Es un buen ejemplo para repasar cardinalidades con el modelo E-R del taller anterior.

2.2 Por qué `clave_hash` y no `clave`. Una contraseña **nunca** se guarda tal cual. Si alguien roba una copia de la base, vería las claves de todos (y muchas personas repiten la misma clave en el correo y el banco). Se guarda un **hash**: el resultado de una función matemática de un solo sentido. Del hash no se puede recuperar la contraseña; para verificarla, se vuelve a calcular y se compara.

Werkzeug, que viene con Flask, hace las dos operaciones. Pruébelo en la terminal con el entorno activo:

```
>>> from werkzeug.security import generate_password_hash, check_password_hash
>>> h = generate_password_hash("Ventas2026*")
>>> h
'scrypt:32768:8:1$Tq4xK...$5f0c2a...'    # cambia cada vez: incluye una "sal"
aleatoria
>>> check_password_hash(h, "Ventas2026*")
True
>>> check_password_hash(h, "otra-clave")
False
```

El texto del hash tiene tres partes separadas por `$`: el algoritmo (`scrypt` y sus parámetros), la **sal** (texto aleatorio que hace que dos usuarios con la misma clave tengan hashes distintos) y el hash propiamente dicho.

2.3 Los usuarios iniciales. El mismo archivo inserta los 8 usuarios con sus hashes ya calculados (por eso las líneas son largas):

```
INSERT INTO usuarios (usuario, clave_hash, rol, id_empleado) VALUES
  ('mprieto', 'scrypt:32768:8:1$...$...', 'ADMIN', 1),
  ('hbustos', 'scrypt:32768:8:1$...$...', 'ADMIN', 2),
  ...
  ('fcuellar', 'scrypt:32768:8:1$...$...', 'VENDEDOR', 8);
```

Los supervisores y la gerente son `ADMIN` (clave `Admin2026*`); los asesores son `VENDEDOR` (clave `Ventas2026*`).

2.4 Permisos del usuario de MySQL. El portal se conecta con el usuario `techandes_api` del taller de la API REST, que solo tiene `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre `techandes`. Como `usuarios` está dentro de esa base, no hace falta otro permiso. Si el usuario no existe, ejecute `sql/02_permisos.sql`:

```
CREATE USER IF NOT EXISTS 'techandes_api'@'localhost' IDENTIFIED BY 'TechAndes2026*';
GRANT SELECT, INSERT, UPDATE, DELETE ON techandes.* TO 'techandes_api'@'localhost';
FLUSH PRIVILEGES;
```

Orden de ejecución en Workbench: `sql/00_techandes_db.sql` (solo si la base no existe o se modificó), luego `01_usuarios.sql` y `02_permisos.sql`. Al final del primero se ve la lista de los 8 usuarios con su empleado.

4. Paso 3: estructura, configuración y conexión a MySQL

El proyecto se organiza por responsabilidades: la conexión en un archivo, la seguridad en otro, cada sección del menú en su propio módulo y el HTML separado del Python. Cree esta estructura de carpetas:

```
techandes-portal/
├── app.py              crea la aplicación y registra los módulos
├── db.py               conexión a MySQL y funciones consultar / ejecutar
├── seguridad.py        decoradores login_requerido y rol_requerido
├── .env                configuración privada (no se sube a GitHub)
├── requirements.txt
├── rutas/              un archivo por sección del menú
│   ├── __init__.py     (vacío: convierte la carpeta en paquete de Python)
│   ├── auth.py         inicio y cierre de sesión
│   ├── inicio.py
│   ├── productos.py
│   ├── clientes.py
│   ├── ventas.py
│   ├── pedidos.py
│   └── informes.py
├── templates/          páginas HTML (Jinja2)
│   ├── base.html       banner + bienvenida + menú (todas las páginas la heredan)
│   ├── login.html
│   ├── inicio.html
│   └── productos/, clientes/, ventas/, pedidos/, informes/, errores/
├── static/
│   ├── css/estilos.css
│   └── img/banner.svg
└── sql/                scripts para Workbench
```

Los nombres `templates` y `static` no son opcionales: Flask busca ahí las plantillas y los archivos estáticos.

3.1 Archivo `.env`

```
SECRET_KEY=cambie-esta-clave-por-una-larga-y-aleatoria
DB_HOST=localhost
DB_PORT=3306
DB_USER=techandes_api
```

```
DB_PASSWORD=TechAndes2026*
DB_NAME=techandes
```

`SECRET_KEY` firma la cookie de sesión: con ella Flask detecta si alguien modificó la cookie para hacerse pasar por otro usuario. Debe ser larga, aleatoria y secreta. Se puede generar con `python -c "import secrets; print(secrets.token_hex(32))"`. El `.env` va en `.gitignore`; en el repositorio se deja `.env.example` sin datos reales.

3.2 `db.py`: una conexión por petición

```
import os

import pymysql
from flask import g


def obtener_conexion():
    """Devuelve la conexión de la petición actual; la crea si no existe."""
    if "db" not in g:
        g.db = pymysql.connect(
            host=os.getenv("DB_HOST", "localhost"),
            port=int(os.getenv("DB_PORT", "3306")),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME", "techandes"),
            charset="utf8mb4",
            cursorclass=pymysql.cursors.DictCursor,  # cada fila llega como
diccionario
            autocommit=True,
        )
    return g.db


def cerrar_conexion(error=None):
    """Flask la llama al final de cada petición."""
    conexion = g.pop("db", None)
    if conexion is not None:
        conexion.close()


def consultar(sql, parametros=None, uno=False):
    """SELECT: lista de diccionarios, o uno solo si uno=True."""
    with obtener_conexion().cursor() as cursor:
        cursor.execute(sql, parametros)
        return cursor.fetchone() if uno else cursor.fetchall()


def ejecutar(sql, parametros=None):
    """INSERT, UPDATE o DELETE: devuelve (id_insertado, filas_afectadas)."""
    with obtener_conexion().cursor() as cursor:
        filas = cursor.execute(sql, parametros)
        return cursor.lastrowid, filas
```

Ideas clave:

1. **`g`** es un objeto de Flask que vive solo durante una petición. La conexión se abre la primera vez que una página la necesita y se cierra al terminar, aunque haya un error.
2. **`DictCursor`** hace que cada fila llegue como diccionario: `fila["nombre"]` en vez de `fila[1]`. El código y las plantillas quedan mucho más legibles.
3. **`autocommit=True`**: cada `INSERT`, `UPDATE` o `DELETE` se guarda de inmediato. Para operaciones de varias sentencias se abre una transacción explícita (paso 8).
4. **Parámetros `%s`, nunca f-strings con datos del usuario.** En PyMySQL el marcador es `%s` (no `?`, como en otros conectores), y vale para cualquier tipo de dato, no solo texto:

```
# MAL: inyección SQL. Si buscar vale  ' OR '1'='1  devuelve todo
consultar(f"SELECT * FROM productos WHERE nombre LIKE '%{buscar}%'")

# BIEN: PyMySQL escapa el valor
consultar("SELECT * FROM productos WHERE nombre LIKE %s", (f"%{buscar}%",))
```

Cuidado con la coma en `(valor,)`: una tupla de un solo elemento la necesita.

El archivo del proyecto tiene además `mensaje_error()`, que traduce los códigos de error de MySQL más comunes: 1062 (dato único repetido), 1451 (registro en uso por una FK), 1452 (FK que apunta a algo inexistente) y 3819 (`CHECK`).

3.3 `app.py`: la fábrica de la aplicación

```
import os
from datetime import date, datetime

from dotenv import load_dotenv
from flask import Flask, render_template

load_dotenv()

from db import cerrar_conexion


def crear_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "clave-solo-para-desarrollo")
    app.teardown_appcontext(cerrar_conexion)

    @app.template_filter("pesos")
    def pesos(valor):
        """1499000 -> '$ 1.499.000'"""
        return "$ " + f"{float(valor or 0):,.0f}".replace(",", ".")

    @app.template_filter("fecha")
    def fecha(valor, con_hora=False):
        if valor is None:
            return ""
        if isinstance(valor, str):
            valor = datetime.fromisoformat(valor)
        formato = "%d/%m/%Y %H:%M" if con_hora and isinstance(valor, datetime) else
"%d/%m/%Y"
        return valor.strftime(formato)
```

```
    @app.context_processor
    def variables_globales():
        return {"anio_actual": date.today().year}

    from rutas.auth import bp as auth_bp
    from rutas.inicio import bp as inicio_bp
    from rutas.productos import bp as productos_bp
    from rutas.clientes import bp as clientes_bp
    from rutas.ventas import bp as ventas_bp
    from rutas.pedidos import bp as pedidos_bp
    from rutas.informes import bp as informes_bp

    for blueprint in (auth_bp, inicio_bp, productos_bp, clientes_bp, ventas_bp,
pedidos_bp, informes_bp):
        app.register_blueprint(blueprint)

    @app.errorhandler(403)
    def prohibido(error):
        return render_template("errores/error.html", codigo=403,
                               mensaje="No tiene permisos para ver esta página."),
403

    @app.errorhandler(404)
    def no_encontrado(error):
        return render_template("errores/error.html", codigo=404,
                               mensaje="La página que busca no existe."), 404

    return app


app = crear_app()

if __name__ == "__main__":
    app.run(debug=True)
```

Conceptos nuevos:

- **Blueprint:** un módulo con sus propias rutas (por ejemplo, todo lo de productos). `app.py` solo los registra. Así varios estudiantes pueden trabajar en módulos distintos sin pisarse.
- **Filtros de plantilla:** funciones que se usan en el HTML con `|`. `{{ 1499000 | pesos }}` muestra `$ 1.499.000`; `{{ pedido.fecha_pedido | fecha(true) }}` muestra `17/02/2025 19:45`.
- **`context_processor`:** variables que llegan a todas las plantillas sin pasarlas una por una.
- **`debug=True`:** reinicia el servidor al guardar un cambio y muestra los errores en detalle. Solo para desarrollo; nunca en producción.

Mientras los módulos de `rutas/` no existan, comente sus líneas `from rutas...` y su entrada en el `for`; se van activando a medida que se crean.

5. Paso 4: la plantilla base (banner, bienvenida y menú)

Las cinco franjas del ejemplo del videoclub (banner, barra de bienvenida, menú, contenido y marco) se escriben **una sola vez** en `templates/base.html`. Cada página del portal hereda esa plantilla y solo define su propio contenido. Si mañana se cambia el menú, cambia en todas las páginas.

4.1 `templates/base.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block titulo %}Portal{% endblock %} · TechAndes</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/estilos.css') }}">
</head>
<body>
  {% set usuario = session.usuario %}
  {% set seccion = request.blueprint %}

  <div class="marco">
    <!-- 1. Banner -->
    <header class="banner">
      <img src="{{ url_for('static', filename='img/banner.svg') }}" alt="TechAndes
S.A.S.">
    </header>

    <!-- 2. Barra de bienvenida -->
    <div class="bienvenida">
      <span>BIENVENIDO(A), <strong>{{ usuario.nombre }}</strong> · {{ usuario.cargo
}}</span>
      <span class="rol">{{ usuario.rol }}</span>
      <a href="{{ url_for('auth.logout') }}">Cerrar sesión</a>
    </div>

    <!-- 3. Menú principal -->
    <nav class="menu">
      <a href="{{ url_for('inicio.inicio') }}" class="{{ 'activo' if seccion ==
'inicio' }}">Inicio</a>
      <a href="{{ url_for('productos.lista') }}" class="{{ 'activo' if seccion ==
'productos' }}">Productos</a>
      <a href="{{ url_for('clientes.lista') }}" class="{{ 'activo' if seccion ==
'clientes' }}">Clientes</a>
      <a href="{{ url_for('ventas.nueva') }}" class="{{ 'activo' if seccion ==
'ventas' }}">Nueva venta</a>
      <a href="{{ url_for('pedidos.lista') }}" class="{{ 'activo' if seccion ==
'pedidos' }}">Pedidos</a>
      {% if usuario.rol == 'ADMIN' %}
        <a href="{{ url_for('informes.index') }}" class="{{ 'activo' if seccion ==
'informes' }}">Informes</a>
      {% endif %}
    </nav>

    <!-- 4. Contenido de cada página -->
    <main class="contenido">
      {% with mensajes = get_flashed_messages(with_categories=true) %}
        {% for categoria, mensaje in mensajes %}
          <div class="mensaje {{ categoria }}">{{ mensaje }}</div>
        {% endfor %}
      {% endwith %}
```

```
      {% block contenido %}{% endblock %}
    </main>

    <!-- 5. Pie de página -->
    <footer class="pie">© {{ anio_actual }} TechAndes S.A.S. · Portal interno de
ventas</footer>
  </div>
</body>
</html>
```

La sintaxis de Jinja2 en tres reglas:

Sintaxis	Para qué	Ejemplo
`{{ ... }}`	Imprimir un valor	`{{ usuario.nombre }}`
`{% ... %}`	Lógica: `if`, `for`, `block`, `set`	`{% if usuario.rol == 'ADMIN' %}`
`{# ... #}`	Comentario que no llega al navegador	`{# pendiente: paginación #}`

Puntos que hay que entender:

1. **`{% block contenido %}`** es un hueco. Cada página hija lo llena con `{% extends "base.html" %}` y su propio `{% block contenido %} ... {% endblock %}`.
2. **`url_for('productos.lista')`** genera la URL a partir del nombre `blueprint.función`. Si la ruta cambia, los enlaces no se rompen. Para archivos estáticos se usa `url_for('static', filename=...)`.
3. **`session.usuario`** contiene los datos que guardó el inicio de sesión (paso 5). Por eso la barra muestra el nombre real del empleado.
4. **Menú según el rol:** *Informes* solo se dibuja para `ADMIN`. Ocultar el enlace no basta como seguridad (alguien puede escribir la URL a mano); por eso la ruta también se protege en Python (paso 5).
5. **Pestaña activa:** `request.blueprint` dice en qué módulo está la página, y esa opción del menú recibe la clase `activo`.
6. **Mensajes flash:** avisos de una sola vez ("Producto creado", "Stock insuficiente") que Python envía con `flash(texto, categoria)` y la plantilla muestra con un color según la categoría (`exito`, `error` o `aviso`).
7. **Escape automático:** Jinja2 convierte `<`, `>` y comillas en entidades HTML. Si un cliente se registra con el nombre `<script>...`, se muestra como texto y no se ejecuta (protección contra XSS).

4.2 Una página hija mínima

```
{% extends "base.html" %}
{% block titulo %}Inicio{% endblock %}

{% block contenido %}
<h1>Panel principal</h1>
<p class="subtitulo">Resumen de la operación comercial de TechAndes.</p>
{% endblock %}
```

4.3 El banner. `static/img/banner.svg` es un dibujo vectorial original: logo de un chip con una montaña, el nombre TECHANDES, el lema y la cordillera de los Andes con trazos de circuitos. Al ser SVG pesa unos 4 KB y se ve nítido en cualquier pantalla. Los estudiantes pueden reemplazarlo por su propio

diseño (Canva, Figma o Inkscape), exportado como `banner.png` o `banner.svg` de 1100 × 220 píxeles. No se deben usar imágenes con derechos de autor sin permiso.

4.4 Los estilos (`static/css/estilos.css`). El archivo completo está en el proyecto. Las reglas que dan la estructura del videoclub son estas:

```
:root {                                  /* colores reutilizables */
  --azul-oscuro: #0b2a4a;
  --azul: #1673b8;
  --cian: #36c2e0;
}

body {                                   /* fondo degradado azul */
  background: radial-gradient(circle at 20% 10%, #8fd3f4 0%, #3b8fd0 40%, #12406f
100%) fixed;
  padding: 24px 16px;
}

.marco {                                 /* recuadro central */
  max-width: 1100px;
  margin: 0 auto;
  background: #f7fbfe;
  border-radius: 14px;
  overflow: hidden;
  box-shadow: 0 18px 50px rgba(5, 25, 50, 0.45);
}

.bienvenida {                            /* franja de bienvenida */
  display: flex; justify-content: center; gap: 14px;
  padding: 10px 16px;
  background: linear-gradient(#eaf6fd, #cfe8f7);
}

.menu {                                  /* barra de menú */
  display: flex; flex-wrap: wrap; justify-content: center;
  background: linear-gradient(#1b78bd, #12599a);
}
.menu a { color: #fff; font-weight: 600; padding: 12px 20px; text-decoration: none; }
.menu a.activo { background: #f7fbfe; color: var(--azul-oscuro); }   /* pestaña
actual */
```

Las **variables CSS** (`--azul`) permiten cambiar la paleta de todo el portal editando una sola línea. `flex-wrap: wrap` hace que el menú pase a dos filas en el celular en lugar de salirse de la pantalla.

6. Paso 5: inicio y cierre de sesión, protección y roles

El inicio de sesión compara la contraseña escrita con el hash guardado en MySQL y, si coincide, guarda los datos del empleado en la **sesión**. A partir de ahí, dos decoradores protegen las páginas: uno exige haber iniciado sesión y otro exige un rol.

5.1 ¿Qué es la sesión? HTTP no recuerda nada entre una petición y la siguiente. Flask guarda los datos de `session` en una **cookie firmada** con la `SECRET_KEY`: el navegador la envía en cada petición y Flask verifica que nadie la haya alterado. Es el equivalente de `$_SESSION` en el ejemplo PHP del videoclub.

5.2 `rutas/auth.py`

```
from flask import Blueprint, flash, redirect, render_template, request, session,
url_for
from werkzeug.security import check_password_hash

from db import consultar, ejecutar

bp = Blueprint("auth", __name__)


@bp.route("/login", methods=["GET", "POST"])
def login():
    if "usuario" in session:
        return redirect(url_for("inicio.inicio"))

    if request.method == "POST":
        usuario = request.form.get("usuario", "").strip().lower()
        clave = request.form.get("clave", "")

        fila = consultar(
            """SELECT u.id_usuario, u.usuario, u.clave_hash, u.rol, u.activo,
                      e.id_empleado, CONCAT(e.nombres, ' ', e.apellidos) AS nombre,
e.cargo
               FROM usuarios u
               JOIN empleados e ON e.id_empleado = u.id_empleado
               WHERE u.usuario = %s""",
            (usuario,),
            uno=True,
        )

        if fila is None or not check_password_hash(fila["clave_hash"], clave):
            flash("Usuario o contraseña incorrectos.", "error")
        elif not fila["activo"]:
            flash("Su usuario está desactivado. Contacte al administrador.", "error")
        else:
            session.clear()
            session["usuario"] = {
                "id_usuario": fila["id_usuario"],
                "usuario": fila["usuario"],
                "rol": fila["rol"],
                "id_empleado": fila["id_empleado"],
                "nombre": fila["nombre"],
                "cargo": fila["cargo"],
            }
            ejecutar("UPDATE usuarios SET ultimo_acceso = NOW() WHERE id_usuario =
%s",
                     (fila["id_usuario"],))

            siguiente = request.args.get("siguiente", "")
            if not siguiente.startswith("/") or siguiente.startswith("//"):
                siguiente = url_for("inicio.inicio")
            return redirect(siguiente)

    return render_template("login.html")
```

```
@bp.route("/logout")
def logout():
    session.clear()
    flash("Sesión cerrada correctamente.", "exito")
    return redirect(url_for("auth.login"))
```

Decisiones de seguridad que conviene discutir en clase:

1. **El mismo mensaje para usuario inexistente y clave incorrecta.** Si dijera "ese usuario no existe", un atacante podría averiguar qué usuarios son reales.
2. **`session.clear()` antes de guardar el usuario.** Evita que se mezclen datos de una sesión anterior en el mismo navegador.
3. **No se guarda la contraseña ni el hash en la sesión**, solo los datos necesarios para mostrar el nombre y decidir permisos.
4. **El parámetro `siguiente`.** Si alguien abre `/productos/` sin sesión, se le envía a `/login?siguiente=/productos/` y, al entrar, vuelve allí. Solo se aceptan rutas internas (que empiecen por una sola `/`): así nadie puede fabricar un enlace que, después del login, lleve a un sitio falso.
5. **Patrón POST → redirect.** Después de un POST exitoso se responde con `redirect` y no con una página. Si el usuario recarga, el navegador repite un GET inofensivo y no reenvía el formulario.

5.3 La página `templates/login.html`. No hereda de `base.html`, porque todavía no hay usuario que mostrar en la barra. Es una tarjeta centrada con el banner, los mensajes flash y el formulario:

```
<form method="post" class="login-cuerpo">
  <h1>Portal de ventas</h1>
  {% with mensajes = get_flashed_messages(with_categories=true) %}
    {% for categoria, mensaje in mensajes %}
      <div class="mensaje {{ categoria }}">{{ mensaje }}</div>
    {% endfor %}
  {% endwith %}
  <label>Usuario
    <input name="usuario" autocomplete="username" required autofocus
           value="{{ request.form.usuario or '' }}">
  </label>
  <label>Contraseña
    <input name="clave" type="password" autocomplete="current-password" required>
  </label>
  <button class="boton" type="submit">Ingresar</button>
</form>
```

Los `name` de los campos (`usuario`, `clave`) son los que Python lee con `request.form.get(...)`. Si el login falla, el usuario escrito se conserva (`value=...`) pero la contraseña no.

5.4 `seguridad.py`: los decoradores

```
from functools import wraps

from flask import abort, flash, redirect, request, session, url_for


def login_requerido(vista):
```

```
    """Solo deja pasar a usuarios que iniciaron sesión."""
    @wraps(vista)
    def envoltura(*args, **kwargs):
        if "usuario" not in session:
            flash("Inicie sesión para continuar.", "aviso")
            return redirect(url_for("auth.login", siguiente=request.path))
        return vista(*args, **kwargs)
    return envoltura


def rol_requerido(*roles):
    """Solo deja pasar a usuarios con alguno de los roles indicados."""
    def decorador(vista):
        @wraps(vista)
        @login_requerido
        def envoltura(*args, **kwargs):
            if session["usuario"]["rol"] not in roles:
                abort(403)
            return vista(*args, **kwargs)
        return envoltura
    return decorador
```

Un **decorador** es una función que envuelve a otra para agregarle comportamiento, en este caso una verificación antes de ejecutar la página. Se usa así:

```
@bp.route("/")
@login_requerido              # cualquier usuario con sesión
def lista(): ...

@bp.route("/nuevo", methods=["GET", "POST"])
@rol_requerido("ADMIN")       # solo administradores
def nuevo(): ...
```

El orden importa: `@bp.route` va siempre primero (arriba). `@wraps` conserva el nombre de la función original; sin él, Flask vería varias rutas llamadas `envoltura` y fallaría.

Matriz de permisos del portal:

Módulo	VENDEDOR	ADMIN
Inicio	Sí (con "Mis ventas")	Sí (con accesos rápidos)
Productos: consultar	Sí	Sí
Productos: crear, editar, eliminar	No (403)	Sí
Clientes: consultar y registrar	Sí	Sí
Nueva venta	Sí, a su nombre	Sí, a su nombre
Pedidos: consultar y cambiar estado	Sí	Sí
Informes	No (403, y no aparece en el menú)	Sí

7. Paso 6: página de inicio con indicadores

La página de inicio es el primer módulo completo: una ruta en Python que hace varias consultas y una plantilla que pinta sus resultados. Muestra cuatro indicadores, las ventas del usuario conectado, los últimos 5 pedidos y los productos con menos de 10 unidades.

6.1 `rutas/inicio.py` (se muestran dos de las cuatro consultas; el archivo completo está en el proyecto):

```
from flask import Blueprint, render_template, session

from db import consultar
from seguridad import login_requerido

bp = Blueprint("inicio", __name__)


@bp.route("/")
@login_requerido
def inicio():
    indicadores = consultar(
        """SELECT
             (SELECT COALESCE(SUM(d.cantidad * d.precio_unitario), 0)
              FROM detalle_pedido d JOIN pedidos p ON p.id_pedido = d.id_pedido
              WHERE p.estado <> 'CANCELADO')                                AS
ventas_totales,
             (SELECT COUNT(*) FROM pedidos WHERE estado = 'PENDIENTE')    AS
pendientes,
             (SELECT COUNT(*) FROM clientes)                              AS
clientes,
             (SELECT COUNT(*) FROM productos WHERE activo = 1 AND stock < 10) AS
stock_bajo""",
        uno=True,
    )

    mis_ventas = consultar(
        """SELECT COUNT(DISTINCT p.id_pedido) AS pedidos,
                  COALESCE(SUM(d.cantidad * d.precio_unitario), 0) AS total
           FROM pedidos p
           JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
           WHERE p.id_empleado = %s AND p.estado <> 'CANCELADO'""",
        (session["usuario"]["id_empleado"],),
        uno=True,
    )

    # ... ultimos (5 pedidos más recientes) y bajo_stock (activos con stock < 10)

    return render_template("inicio.html", indicadores=indicadores,
mis_ventas=mis_ventas,
                           ultimos=ultimos, bajo_stock=bajo_stock)
```

`render_template` recibe el nombre de la plantilla y las variables que esta podrá usar. "Mis ventas" filtra por `session["usuario"]["id_empleado"]`: cada asesor ve sus propias cifras sin elegir nada.

6.2 `templates/inicio.html` (fragmentos):

```
{% extends "base.html" %}
{% block titulo %}Inicio{% endblock %}

{% block contenido %}
<h1>Panel principal</h1>

<div class="indicadores">
  <div class="indicador"><span>Ventas efectivas</span><strong>{{
indicadores.ventas_totales | pesos }}</strong></div>
  <div class="indicador"><span>Pedidos pendientes</span><strong>{{
indicadores.pendientes }}</strong></div>
  <div class="indicador"><span>Clientes registrados</span><strong>{{
indicadores.clientes }}</strong></div>
  <div class="indicador alerta"><span>Productos con stock bajo</span><strong>{{
indicadores.stock_bajo }}</strong></div>
</div>

<table>
  <thead><tr><th>#</th><th>Fecha</th><th>Cliente</th><th
class="num">Total</th><th>Estado</th></tr></thead>
  <tbody>
    {% for p in ultimos %}
    <tr>
      <td><a href="{{ url_for('pedidos.detalle', id_pedido=p.id_pedido) }}">{{
p.id_pedido }}</a></td>
      <td>{{ p.fecha_pedido | fecha }}</td>
      <td>{{ p.cliente }}</td>
      <td class="num">{{ p.total | pesos }}</td>
      <td><span class="estado {{ p.estado }}">{{ p.estado }}</span></td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% endblock %}
```

El `{% for %}` recorre la lista de diccionarios que devolvió `consultar`. La clase `estado {{ p.estado }}` produce, por ejemplo, `class="estado PENDIENTE"`, y el CSS le da a cada estado su color. Un `{% for %}` puede tener un `{% else %}` que se muestra cuando la lista está vacía; se usa en la tabla de stock bajo.

Resultado esperado con los 200 registros originales: ventas efectivas **$ 66.538.300**, **2** pedidos pendientes, **40** clientes y **8** productos con stock bajo. Si el usuario es `klozada`, "Mis ventas" muestra **12** pedidos por **$ 20.828.100**, la misma cifra de Kevin Lozada en el ranking A4 del taller de SQL.

8. Paso 7: módulo de productos (CRUD)

Productos es el módulo modelo: listar con filtros, crear, editar y eliminar. Los demás módulos repiten este mismo patrón. Todos los usuarios consultan; solo `ADMIN` modifica.

Acción	Ruta	Método	Plantilla
Listar y filtrar	`/productos/?buscar=...&categoria=...`	GET	`productos/lista.html`

Acción	Ruta	Método	Plantilla
Mostrar formulario vacío / guardar nuevo	`/productos/nuevo`	GET / POST	`productos/formulario.html`
Mostrar formulario lleno / guardar cambios	`/productos/16/editar`	GET / POST	`productos/formulario.html`
Eliminar	`/productos/16/eliminar`	POST	(redirige a la lista)

7.1 Listar con filtros

```
bp = Blueprint("productos", __name__, url_prefix="/productos")


@bp.route("/")
@login_requerido
def lista():
    buscar = request.args.get("buscar", "").strip()
    categoria = request.args.get("categoria", "")

    sql = """SELECT p.id_producto, p.nombre, p.precio, p.stock, p.activo, c.nombre AS
categoria
             FROM productos p
             JOIN categorias c ON c.id_categoria = p.id_categoria
             WHERE 1 = 1"""
    parametros = []
    if buscar:
        sql += " AND p.nombre LIKE %s"
        parametros.append(f"%{buscar}%")
    if categoria.isdigit():
        sql += " AND p.id_categoria = %s"
        parametros.append(categoria)
    sql += " ORDER BY c.nombre, p.nombre"

    productos = consultar(sql, parametros)
    return render_template("productos/lista.html", productos=productos,
categorias=categorias(),
                           buscar=buscar, categoria=categoria)
```

- `url_prefix="/productos"` hace que todas las rutas del blueprint empiecen por `/productos`.
- `request.args` contiene los parámetros de la URL (lo que va después del `?`). El formulario de filtros usa `method="get"`, así los filtros quedan en la URL y se pueden compartir o guardar como favorito.
- `WHERE 1 = 1` permite agregar cada filtro con `AND` sin preguntar si es el primero. Cada valor va como `%s`, nunca pegado al SQL.

En la plantilla, los botones de modificar solo se dibujan para el administrador:

```
{% for p in productos %}
<tr>
  <td>{{ p.id_producto }}</td>
  <td>{{ p.nombre }}</td>
  <td>{{ p.categoria }}</td>
```

```
  <td class="num">{{ p.precio | pesos }}</td>
  <td class="num">{{ p.stock }}</td>
  <td>{% if p.activo %}Activo{% else %}<span class="inactivo">Descontinuado</span>{%
endif %}</td>
  {% if session.usuario.rol == 'ADMIN' %}
  <td class="acciones">
    <a class="boton mini" href="{{ url_for('productos.editar',
id_producto=p.id_producto) }}">Editar</a>
    <form method="post" action="{{ url_for('productos.eliminar',
id_producto=p.id_producto) }}"
          onsubmit="return confirm('¿Eliminar {{ p.nombre }}?');">
      <button class="boton mini peligro" type="submit">Eliminar</button>
    </form>
  </td>
  {% endif %}
</tr>
{% else %}
<tr><td colspan="7" class="vacio">No hay productos con esos filtros.</td></tr>
{% endfor %}
```

Eliminar siempre con POST, nunca con un enlace. Un enlace (`<a href="...">`) es un GET, y los navegadores, buscadores o extensiones pueden abrir enlaces por su cuenta. Las acciones que cambian datos van en un formulario `method="post"`.

7.2 Validar el formulario

```
def leer_formulario():
    """Lee y valida el formulario. Devuelve (datos, lista_de_errores)."""
    datos = {
        "nombre": request.form.get("nombre", "").strip(),
        "id_categoria": request.form.get("id_categoria", ""),
        "precio": request.form.get("precio", ""),
        "stock": request.form.get("stock", "0"),
        "activo": 1 if request.form.get("activo") else 0,
    }
    errores = []
    if not datos["nombre"]:
        errores.append("El nombre es obligatorio.")
    elif len(datos["nombre"]) > 100:
        errores.append("El nombre no puede superar 100 caracteres.")
    if not datos["id_categoria"].isdigit():
        errores.append("Seleccione una categoría.")
    try:
        if float(datos["precio"]) <= 0:
            errores.append("El precio debe ser mayor que 0.")
    except ValueError:
        errores.append("El precio debe ser un número.")
    if not datos["stock"].isdigit():
        errores.append("El stock debe ser un número entero mayor o igual a 0.")
    return datos, errores
```

Se valida en Python aunque el HTML ya tenga `required` y `min="1"`: esas reglas del navegador se pueden saltar fácilmente (por ejemplo con las herramientas de desarrollo). Y la base de datos tiene sus `CHECK` como última línea de defensa. Un checkbox desmarcado **no se envía**; por eso `activo` se calcula según exista o no en `request.form`.

7.3 Crear: GET muestra, POST guarda

```
@bp.route("/nuevo", methods=["GET", "POST"])
@rol_requerido("ADMIN")
def nuevo():
    datos = {"nombre": "", "id_categoria": "", "precio": "", "stock": "0", "activo":
1}
    if request.method == "POST":
        datos, errores = leer_formulario()
        if not errores:
            try:
                ejecutar(
                    """INSERT INTO productos (nombre, id_categoria, precio, stock,
activo)
                       VALUES (%s, %s, %s, %s, %s)""",
                    (datos["nombre"], datos["id_categoria"], datos["precio"],
datos["stock"], datos["activo"]),
                )
                flash(f"Producto «{datos['nombre']}» creado.", "exito")
                return redirect(url_for("productos.lista"))
            except pymysql.MySQLError as error:
                errores.append(mensaje_error(error))
        for e in errores:
            flash(e, "error")
    return render_template("productos/formulario.html", datos=datos,
categorias=categorias(),
                           titulo="Nuevo producto")
```

El flujo de un formulario en Flask:

1. El usuario abre `/productos/nuevo` (GET) y ve el formulario vacío.
2. Lo envía (POST a la misma URL). Si hay errores, se vuelve a mostrar **con lo que escribió** (`datos`) y los mensajes en rojo, para que no tenga que llenarlo de nuevo.
3. Si todo está bien, se inserta, se agrega un mensaje flash y se **redirige** a la lista.

7.4 Editar usa la misma plantilla. En el GET se carga el producto con `SELECT * FROM productos WHERE id_producto = %s` (si no existe, `abort(404)`) y se pasa como `datos`; en el POST se ejecuta un `UPDATE`. La plantilla marca la categoría actual:

```
<select name="id_categoria" required>
  <option value="">Seleccione…</option>
  {% for c in categorias %}
    <option value="{{ c.id_categoria }}"
      {{ 'selected' if c.id_categoria|string == datos.id_categoria|string }}>{{
c.nombre }}</option>
  {% endfor %}
</select>
```

El `|string` es necesario: desde MySQL el id llega como número (5) y desde el formulario como texto ("5"), y `5 == "5"` es falso.

7.5 Eliminar y el error 1451

```
@bp.route("/<int:id_producto>/eliminar", methods=["POST"])
@rol_requerido("ADMIN")
def eliminar(id_producto):
```

```
    try:
        _, filas = ejecutar("DELETE FROM productos WHERE id_producto = %s",
(id_producto,))
        if filas == 0:
            abort(404)
        flash("Producto eliminado.", "exito")
    except pymysql.MySQLError as error:
        if error.args[0] == 1451:
            flash("Este producto ya tiene ventas y no se puede eliminar. "
                  "Desmarque «Activo» para retirarlo del catálogo.", "error")
        else:
            flash(mensaje_error(error), "error")
    return redirect(url_for("productos.lista"))
```

`<int:id_producto>` toma el número de la URL y lo convierte en entero; `/productos/abc/eliminar` da 404 automáticamente. El error 1451 es la llave foránea con `ON DELETE RESTRICT` del taller de SQL: intente eliminar el mouse Logitech (id 16) y verá el mensaje. La solución de negocio es desmarcar *Activo* en lugar de borrar.

9. Paso 8: ventas, pedidos, clientes e informes

Los cuatro módulos restantes repiten el patrón de productos y agregan dos ideas nuevas: la **transacción** para registrar una venta y la devolución de stock al cancelar un pedido. Aquí se explica lo nuevo; el código completo está comentado en el proyecto.

8.1 Nueva venta (`rutas/ventas.py`)

El formulario tiene el cliente, el método de pago y 5 filas de producto y cantidad (las filas vacías se ignoran). El vendedor no se elige: es el usuario conectado. Una venta toca tres tablas, así que todo va en una transacción:

```
class ErrorVenta(Exception):
    """Error de negocio: aborta la transacción con un mensaje claro."""


def registrar_pedido(id_cliente, metodo_pago, items):
    """Inserta pedido + detalle y descuenta stock. Todo o nada."""
    conexion = obtener_conexion()
    conexion.begin()                    # START TRANSACTION
    try:
        with conexion.cursor() as cursor:
            cursor.execute(
                """INSERT INTO pedidos (id_cliente, id_empleado, estado, metodo_pago)
                   VALUES (%s, %s, 'PENDIENTE', %s)""",
                (id_cliente, session["usuario"]["id_empleado"], metodo_pago),
            )
            id_pedido = cursor.lastrowid

            for id_producto, cantidad in items.items():
                cursor.execute(
                    "SELECT nombre, precio, stock, activo FROM productos WHERE
id_producto = %s FOR UPDATE",
                    (id_producto,),
                )
```

```
                producto = cursor.fetchone()
                if producto is None or not producto["activo"]:
                    raise ErrorVenta(f"El producto {id_producto} no está
disponible.")
                if producto["stock"] < cantidad:
                    raise ErrorVenta(f"Stock insuficiente de {producto['nombre']}: "
                                     f"hay {producto['stock']}, se piden
{cantidad}.")

                cursor.execute(
                    """INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad,
precio_unitario)
                       VALUES (%s, %s, %s, %s)""",
                    (id_pedido, id_producto, cantidad, producto["precio"]),
                )
                cursor.execute(
                    "UPDATE productos SET stock = stock - %s WHERE id_producto = %s",
                    (cantidad, id_producto),
                )
        conexion.commit()               # todo salió bien
        return id_pedido
    except Exception:
        conexion.rollback()             # algo falló: se deshace TODO
        raise
```

Puntos clave:

1. **`conexion.begin()`** abre una transacción aunque la conexión tenga `autocommit=True`. Nada se guarda hasta el `commit()`.
2. **Una excepción propia (`ErrorVenta`)** separa los errores de negocio ("stock insuficiente") de los errores técnicos. La ruta la atrapa y la muestra con `flash`; el `except` de `registrar_pedido` ya hizo el `rollback()` antes de relanzarla.
3. **`FOR UPDATE`** bloquea la fila del producto: si dos asesores venden la última unidad al mismo tiempo, el segundo espera y luego ve el stock ya descontado.
4. **El precio lo pone el servidor**, desde la base, en el momento de la venta. El formulario solo envía id y cantidad.
5. **Productos repetidos:** si el usuario elige el mismo producto en dos filas, `leer_lineas()` suma las cantidades en un diccionario `{id_producto: cantidad}`, porque la llave primaria de `detalle_pedido` no permite repetirlo.

Si todo sale bien, la ruta redirige al detalle del pedido nuevo con el mensaje *"Venta registrada: pedido #51"*.

Prueba de atomicidad: registre una venta con una memoria USB (id 25) y 50 portátiles ASUS (id 3, que solo tiene 5). Aparece *"Stock insuficiente de Portátil ASUS TUF Gaming F15: hay 5, se piden 50"*, y en Workbench se comprueba que no se creó el pedido y que la memoria USB conserva sus 80 unidades.

8.2 Pedidos y cancelación (`rutas/pedidos.py`)

La lista filtra por estado y el detalle muestra encabezado, cliente, productos y total. El cambio de estado aplica tres reglas de negocio, también dentro de una transacción:

```
if actual == nuevo:
    flash("El pedido ya tiene ese estado.", "aviso")
elif actual == "CANCELADO":
    flash("Un pedido cancelado no se puede modificar.", "error")
else:
    if nuevo == "CANCELADO":
        # Devolver al inventario las unidades de cada producto del pedido
        cursor.execute("SELECT id_producto, cantidad FROM detalle_pedido WHERE
id_pedido = %s",
                       (id_pedido,))
        for linea in cursor.fetchall():
            cursor.execute("UPDATE productos SET stock = stock + %s WHERE id_producto
= %s",
                           (linea["cantidad"], linea["id_producto"]))
    cursor.execute("UPDATE pedidos SET estado = %s WHERE id_pedido = %s", (nuevo,
id_pedido))
    conexion.commit()
```

Es el equivalente de "Devoluciones" en el videoclub: al cancelar, el inventario vuelve a cuadrar. El estado actual se lee con `SELECT ... FOR UPDATE` para que dos usuarios no cancelen el mismo pedido a la vez y devuelvan el stock dos veces.

8.3 Clientes (`rutas/clientes.py`)

- **Lista** con búsqueda por nombre, apellido o documento y filtro por ciudad. Usa `LEFT JOIN pedidos` con `COUNT` para mostrar cuántos pedidos tiene cada cliente, incluidos los que tienen cero.
- **Nuevo cliente:** valida documento numérico, correo con una expresión regular y ciudad de la lista. Si el documento o el correo ya existen, MySQL responde con el error 1062 (`UNIQUE`) y la página muestra *"Ya existe un cliente con ese documento o correo"*.
- **Detalle** (`/clientes/6`): datos del cliente, total comprado (sin cancelados) e historial de pedidos con enlace a cada uno.

8.4 Informes (`rutas/informes.py`, solo ADMIN)

Cuatro tablas que reutilizan consultas del taller de SQL:

Informe	Consulta de origen	Resultado con los datos originales
Ventas por categoría, con barra de porcentaje	A9	Portátiles 35,4 % y Celulares 35,3 % suman más de dos tercios
Ranking de vendedores con `RANK() OVER`	A4	1.º Brayan Oviedo Calderón, $ 21.491.600
Top 5 productos por unidades	A5 simplificada	1.º Mouse Logitech M280, 10 unidades
Clientes sin compras con `NOT EXISTS`	A1	8 clientes, candidatos para una campaña

El porcentaje de cada categoría se calcula en Python (total de la categoría ÷ gran total × 100) y la plantilla lo usa como ancho de la barra: `<div style="width: {{ f.porcentaje }}%"></div>`. Es un gráfico de barras hecho solo con HTML y CSS.

10. Paso 9: ejecutar y probar el portal

Con el entorno activo y MySQL encendido, el portal se inicia con un solo comando y se prueba siguiendo un guion con resultados conocidos.

9.1 Ejecutar

```
# dentro de techandes-portal, con (.venv) activo
python app.py
```

La terminal muestra `Running on http://127.0.0.1:5000`. Abra esa dirección: el portal lleva a `/login`. Para detener el servidor: `Ctrl + C`. Con `debug=True`, cada vez que se guarda un archivo `.py` el servidor se reinicia solo; los cambios en plantillas y CSS se ven al recargar la página (`Ctrl + F5` si el navegador guardó la versión anterior).

9.2 Guion de pruebas (sobre los 200 registros originales, en este orden):

#	Usuario	Acción	Resultado esperado
1	(ninguno)	Abrir `http://127.0.0.1:5000/productos/`	Redirige a `/login` con el aviso "Inicie sesión para continuar"
2	`klozada`	Ingresar con una clave equivocada	"Usuario o contraseña incorrectos"
3	`klozada`	Ingresar con `Ventas2026*`	Vuelve a Productos; la barra dice Kevin Lozada Perdomo · VENDEDOR
4	`klozada`	Ver el menú	No aparece *Informes* ni los botones Editar / Eliminar
5	`klozada`	Escribir a mano `/informes/`	Error 403
6	`klozada`	Inicio	$ 66.538.300 en ventas; "Mis ventas": 12 pedidos por $ 20.828.100
7	`klozada`	Clientes → Ibagué	10 clientes
8	`klozada`	Nuevo cliente con el correo `andres.gomez1@correo.com`	"Ya existe un cliente con ese documento o correo"
9	`klozada`	Nueva venta: cliente Tomás Ortega, 2 mouse Logitech + 1 SSD Kingston	Pedido #51 por $ 427.000, a nombre de Kevin
10	`klozada`	Nueva venta con 1 memoria USB y 50 portátiles ASUS	"Stock insuficiente..." y no se crea ningún pedido
11	`klozada`	Pedido #51 → CANCELADO	"Las unidades volvieron al inventario"; el mouse vuelve a 60 unidades

#	Usuario	Acción	Resultado esperado
12	`klozada`	Pedido #51 → ENVIADO	"Un pedido cancelado no se puede modificar"
13	`klozada`	Cerrar sesión y luego pulsar *Atrás* y recargar	Vuelve a pedir inicio de sesión
14	`mprieto`	Ingresar con `Admin2026*`	Aparece *Informes* y los botones de administración
15	`mprieto`	Nuevo producto con nombre vacío y precio −5	Dos mensajes de error y el formulario conserva los datos
16	`mprieto`	Crear, editar y eliminar "Hub USB-C 7 en 1"	Tres mensajes de éxito
17	`mprieto`	Eliminar el mouse Logitech (id 16)	"Este producto ya tiene ventas y no se puede eliminar"
18	`mprieto`	Informes	Ranking encabezado por Brayan Oviedo; 8 clientes sin compras

Verificaciones en Workbench después de las pruebas 9 a 11:

```
SELECT * FROM pedidos ORDER BY id_pedido DESC LIMIT 2;      -- el #51, con estado
CANCELADO
SELECT * FROM detalle_pedido WHERE id_pedido = 51;          -- 2 líneas con el precio
del momento
SELECT id_producto, stock FROM productos WHERE id_producto IN (16, 23, 25);
SELECT usuario, ultimo_acceso FROM usuarios;                -- fecha del último
ingreso
```

El número del pedido (#51) supone que nadie ha registrado ventas antes. Para repetir el guion desde cero, reimporte `sql/00_techandes_db.sql` y vuelva a ejecutar `sql/01_usuarios.sql` (el primero borra la base completa, incluida la tabla de usuarios).

9.3 Herramientas de desarrollo (F12). En la pestaña *Red* (*Network*) se ve que el login es un `POST /login` que responde **302** (redirección), y en *Aplicación* → *Cookies* aparece la cookie `session`. Intente modificarla a mano: Flask detecta que la firma no coincide y la descarta, así que el usuario queda sin sesión.

11. Errores frecuentes

Con `debug=True`, Flask muestra en el navegador el error completo con la línea exacta; la terminal también lo imprime. Lea siempre la **última línea** del mensaje: ahí está la causa.

Síntoma	Causa probable	Solución
`'python' no se reconoce como un comando`	No se marcó *Add python.exe to PATH* al instalar	Reinstale marcando la opción, o use `py` en lugar de `python` en Windows

Síntoma	Causa probable	Solución
`ModuleNotFoundError: No module named 'flask'`	El entorno virtual no está activo o no se instaló `requirements.txt`	Active `.venv` y ejecute `pip install -r requirements.txt`
`ModuleNotFoundError: No module named 'rutas'`	Se ejecutó `python app.py` desde otra carpeta, o falta `rutas/__init__.py`	Ejecute desde la carpeta del proyecto y cree el archivo vacío
`RuntimeError: 'cryptography' package is required`	MySQL 8 usa `caching_sha2_password`	`pip install cryptography`
`OperationalError: (1045, "Access denied ...")`	Usuario o clave del `.env` incorrectos	Revise `.env` y `sql/02_permisos.sql`
`OperationalError: (2003, "Can't connect ...")`	MySQL apagado o en otro puerto	Inicie MySQL y revise `DB_PORT`
`ProgrammingError: (1146, "Table 'techandes.usuarios' doesn't exist")`	No se ejecutó `01_usuarios.sql`, o se reimportó la base después	Ejecute `sql/01_usuarios.sql` en Workbench
"Usuario o contraseña incorrectos" con datos correctos	Los hashes de la tabla no corresponden, o se escribió la clave del rol equivocado	Use `Admin2026*` para ADMIN y `Ventas2026*` para VENDEDOR; vuelva a ejecutar `01_usuarios.sql`
`TemplateNotFound: productos/lista.html`	La plantilla no está dentro de `templates/` o el nombre no coincide	Revise la ruta y las mayúsculas del archivo
`BuildError: Could not build url for endpoint 'productos.lista'`	`url_for` usa un nombre que no existe o el blueprint no se registró	Revise el nombre `blueprint.función` y el `for` de `app.py`
`AssertionError: View function mapping is overwriting an existing endpoint`	Falta `@wraps(vista)` en un decorador, o dos funciones se llaman igual	Agregue `@wraps` y use nombres distintos
`TypeError: not all arguments converted during string formatting`	Hay más `%s` o más parámetros de la cuenta, o falta la coma en `(valor,)`	Cuente los `%s` y los valores; use tuplas
`ValueError: unsupported format character`	Un `%` literal en el SQL (por ejemplo `DATE_FORMAT(fecha, '%Y')`) junto con parámetros	Escriba `%%Y` cuando la consulta también tiene `%s`

Síntoma	Causa probable	Solución
El estilo o el banner no cargan (404 en la consola)	Los archivos no están en `static/css` y `static/img`	Revise las carpetas; use `url_for('static', ...)`
Se ve la página pero sin datos nuevos	El navegador usó la versión guardada	Recargue con `Ctrl + F5`
`Address already in use` / puerto 5000 ocupado	Otra instancia del servidor sigue abierta (en macOS, *AirPlay Receiver* usa el 5000)	Cierre la otra terminal o use `app.run(debug=True, port=5001)`

12. Taller y entregables

Con el portal base funcionando, cada equipo lo personaliza y lo extiende. Los retos están ordenados por dificultad.

#	Reto	Qué debe hacer	Concepto que practica
R1	Identidad propia	Diseñar su propio banner (1100 × 220) y cambiar la paleta con las variables CSS	Plantilla base, CSS
R2	Módulo de categorías	CRUD completo de categorías, solo para ADMIN	Patrón CRUD, `UNIQUE`, error 1451
R3	Editar clientes	Ruta `/clientes/<id>/editar` reutilizando el formulario	`UPDATE`, error 1062
R4	Paginación de pedidos	Mostrar 15 pedidos por página con botones *Anterior* y *Siguiente*	`LIMIT` y `OFFSET`
R5	Cambio de contraseña	Página *Mi cuenta* donde el usuario cambia su clave (pide la actual y la nueva dos veces)	`check_password_hash`, `generate_password_hash`
R6	Gestión de usuarios	Pantalla para que el ADMIN cree usuarios, asigne rol y los active o desactive	Relación 1:1 con empleados
R7	Ventas por mes	Informe con el total de cada mes del año elegido	`DATE_FORMAT`, `GROUP BY`; cuidado con `%%`
R8	Exportar a CSV	Botón en Informes que descarga las ventas por categoría como archivo `.csv`	Módulo `csv` y `Response` de Flask
R9	Protección CSRF	Agregar Flask-WTF para que los formularios POST lleven un token	Seguridad web

#	Reto	Qué debe hacer	Concepto que practica
R10	Bloqueo por intentos	Desactivar temporalmente un usuario tras 5 intentos fallidos	Columna nueva en `usuarios`, reglas de negocio

Entregables:

- ☐ Repositorio en GitHub con el proyecto, sin `.venv` ni `.env` (con `.env.example` y `requirements.txt`)
- ☐ `README.md` con instalación, usuarios de prueba y lista de módulos
- ☐ Scripts SQL de los cambios hechos a la base (tablas o columnas nuevas)
- ☐ Capturas de cada módulo, incluida una del error 403 al entrar como VENDEDOR a *Informes*
- ☐ Video corto (máximo 5 minutos) que muestre el inicio de sesión con ambos roles, una venta exitosa, una venta rechazada por stock y la cancelación de un pedido con devolución del inventario

Vista del portal terminado, página de inicio con la sesión de la gerente:

